

PinIT Python Month 1: Lesson Visuals Pilot (Days 1 to 3)

Course: Python Month 1 certification course (quest prefix `python`, file `src/lib/data/pythonLongLessons.ts`)

Scope: only the three lectures of Day 1 (Your First Python Program), Day 2 (Variables and Data Types) and Day 3 (Working With Text). That is 18 lesson parts. No other course, day or file is in scope.

Builder: Antigravity. **Reviewer:** Claude. **Approver:** the owner.

Version: 1.1, 5 October 2026 (1.1: underline, space-dot and avatar-button rules tightened after the first screenshots)

Contents

PinIT Python Month 1: Lesson Visuals Pilot (Days 1 to 3)

- Part A. For the owner (read this page only)
- Part B. Lesson mistakes to fix first
- Part C. What to build (only what Days 1 to 3 need)
- Part D. The 18 pictures, step by step
- Part E. Tasks for Antigravity (in this order)
- Part F. What Claude will check when Antigravity says "done"

Part A. For the owner (read this page only)

A1. What the student will see

- **Left side (45%):** one simple picture for the part being taught, with a one-line caption under it and Back and Next buttons.
- **Right side (55%):** the lesson text, code, "Your turn" and quiz, exactly as today.
- **Avatar:** a small frame at the bottom right that the student can minimise. Today the avatar stands in the left column, so it must move.
- **On a phone:** the picture is on top, the lesson below, and the avatar is a small bubble.

A2. How the picture and the lesson stay together

1. **The voice moves the picture.** When the avatar starts a certain sentence, the picture moves to the matching step.
2. **Words and picture point at each other.** Tap a word such as `balance` in the text, and the `balance` box on the left lights up. Tap the box, and the word lights up.
3. **The picture never lies.** Every number or text in a picture is checked by a test against what Python really prints.
4. **The student is in control.** Nothing loops or plays by itself. Back and Next move one step at a time.

A3. What is in this pilot

- **18 parts, 18 pictures**, each specified step by step in Part D.
- **5 picture types:** flow, boxes, table, letters-in-a-row, side-by-side.
- **Every number and text in the 18 pictures was already checked by Claude in real Python 3 (28 checks, all passed).** The test in Part C repeats these checks automatically.
- **2 mistakes found in the current lessons** (Part B). These are fixed first, because a picture built on a wrong sentence would teach the mistake.

A4. How you check it (no coding needed)

1. Open Day 1, Day 2 and Day 3 of Python Month 1.
2. In every part, press Play and watch: the picture should change when the avatar reaches the matching sentence.
3. Look at each picture for 3 seconds. If you cannot tell what it shows, write down the part number. That picture fails.
4. Tap two words in the text in each part. The matching shape should light up.
5. Open the same lesson on a phone. The picture must be on top and readable.
6. Antigravity must show you the test result line from task V-09: `lesson visuals: all checks passed`.

Part B. Lesson mistakes to fix first

These were found by running the lesson code with real Python 3, the same way the student's browser does.

B1 (C-01). Day 1, Part 3 "How Python reads your code": wrong about missing quotes

What the lesson says:

- Line 2 of the spoken text: "If a line has a mistake, Python stops at that line... The lines above it already ran."
- "Your turn": remove the closing quote on line 2, run it, and see the error.
- Quiz: "Python finds a mistake on line 3 of a 5-line program. What happens?" Correct answer: "Lines 1 and 2 run".

What really happens: a missing quote is a `SyntaxError`. Python checks the whole file before running anything, so **nothing runs, not even line 1**. "Lines above already ran" is only true for mistakes found while running, such as a misspelled name (`NameError`). A student who follows "Your turn" sees no output at all, which contradicts what they were just told.

Fix (replace the text exactly):

- Spoken line 2: "If a line has a mistake that Python finds while running, such as a misspelled word, Python stops at that line and shows an error message. The lines above it already ran, but the lines below it never run. Beginners often think the whole program is broken, when really only one line has a problem."
- Spoken line 4: "Some mistakes are different. If you forget a closing quote or a closing bracket, Python cannot even read the program, so nothing runs at all, not even line 1. This is called a `SyntaxError`. If you see `SyntaxError`, check your quotes and brackets first."
- "Your turn": "Change line 2 to `print("Step 2: add an expense")` and run it. Line 1 prints, then Python stops with a `NameError` on line 2, and line 3 never runs. Fix it, then remove the closing quote on line 2 and run again: this time nothing prints at all, because a `SyntaxError` stops Python before it starts."
- Quiz question: "Line 3 of a 5-line program says `print("Hi")`, a misspelled print. What happens?" Keep the 3 options and keep answer 0. In `why`, add the sentence: "A missing quote would be different: that is a `SyntaxError`, and nothing would run."

Proof (real Python 3): with the closing quote removed on line 2, the output is only `SyntaxError: unterminated string literal (detected at line 2)`, and line 1 does not print. With `Print` on line 2, line 1 prints, and then `NameError: name 'Print' is not defined` appears.

B2 (C-02). Day 2, Part 4 "True and False": "every answer flips" is false

What the lesson says: "Your turn": change `amount = 1200` to `amount = 800`, and "Every answer flips."

What really happens: with 800 the four answers are `False`, `False`, `True`, `Big expense? False`. Only 2 of the 4 change: `amount == 500` stays `False` and `amount != 500` stays `True`. A student who predicts "all flip", as told, is marked wrong by their own run.

Fix: replace "Every answer flips." with "Two of the answers change and two stay the same. Predict which ones before you press Run Code."

B3. After the fixes

The existing test `tests/python_long_lessons.test.ts` must still pass. Part 3's code does not change, only the spoken text, "Your turn" and the quiz, so no output changes.

Part C. What to build (only what Days 1 to 3 need)

C1. The five picture templates

Template	What it draws	Used in parts
flow	2 to 5 boxes in a row joined by arrows. The lit boxes and arrows show what is happening now	1.1, 2.5, 3.1, 3.4, 3.6
boxes	named boxes, each holding one value (a variable and its value)	2.1, 2.2, 2.3, 2.6
table	2 columns, filled row by row. Each row has a colour	1.2, 1.3, 1.5, 1.6, 2.4, 3.5
letters	one cell per character with its position number under it, a pointer, and a highlighted range	3.2, 3.3
compare	two panels side by side, each with code and a result	1.4

Drawing rules for every template:

- Plain HTML boxes and CSS (grid or flex, borders, background colours). Arrows may use a small inline SVG. No canvas, images, video, icon fonts, emoji or animation libraries.
- **Every step stores the whole picture, not a change.** Back and Next simply show step N-1 and N+1, so a picture can never get out of sync.
- When the step changes, only what changed glows for 300 ms (a CSS `transition` on background, border and opacity), then everything stays still. Nothing moves in a loop.
- Under `@media (prefers-reduced-motion: reduce)`, no transitions at all.
- Spaces are drawn as a faint `·` **only** in `flow` and `boxes` values of parts 3.1, 3.4 and 3.6, where the spaces are the lesson. Everywhere else, spaces are normal spaces.
- Code and values use the app's monospace font. Captions use the normal font, at least 15 px.
- The caption has `aria-live="polite"`, so screen readers read each new step.

C2. Only four colours, each with one meaning

Tone	Colour token (already in the app)	Meaning
data	<code>var(--accent)</code>	the thing being looked at now
ok	<code>var(--success)</code>	ran, correct, result
error	<code>var(--coral)</code>	error or wrong
idle	<code>var(--border)</code> with muted text	skipped, not run, not used yet

No other colours. The word-tap highlight uses `data`, not a new colour.

C3. The data format

Create `src/lib/types/lessonVisual.ts`:

```
export type VisualAt = 'say1' | 'say2' | 'say3' | 'say4' | 'example' | 'tryIt';
export type VisualTone = 'data' | 'ok' | 'error' | 'idle';

interface StepBase {
  at: VisualAt;           // which spoken piece starts this step
  caption: string;        // one sentence, at most 80 characters
  checks?: string[];      // values that must appear in the real output (see C6, rule 6)
  whatIf?: string;        // optional changed code; checks are then run against this code
  instead
  mustNotShow?: string[]; // values that must NOT appear in that output
  lastLine?: string;      // the last output line must equal this
}
interface Node { id: string; label: string; tappable?: boolean } // tappable defaults to true

export interface FlowStep extends StepBase {
  values: Record<string, string>; // node id -> text shown in the box
  tones: Record<string, VisualTone>; // node id -> colour
  arrows: [string, string][]; // lit arrows, as [fromId, toId]
}
export interface BoxesStep extends StepBase {
  values: Record<string, string>; // box id -> value ('' = empty)
  tones: Record<string, VisualTone>;
  types?: Record<string, string>; // optional small type tag: 'str', 'int', 'float', 'bool'
}
export interface TableStep extends StepBase {
  rows: { cells: [string, string]; tone: VisualTone }[];
}
export interface LettersStep extends StepBase {
  pointer?: number; // index; negative counts from the end; out of range
= drawn after the last cell in red
  range?: [number, number]; // [start, stop): stop not included
  result: string; // shown under the cells
  tone: VisualTone;
}
export interface ComparePanel { code: string; result: string; tone: VisualTone; checks: string[]; whatIf?: string }
export interface CompareStep extends Omit<StepBase, 'checks' | 'whatIf'> {
  left: ComparePanel; // each panel is checked on its own: its checks run against
  right: ComparePanel; // its whatIf code if given, otherwise against the part's code
}

export type LessonVisual =
  | { template: 'flow'; title: string; nodes: Node[]; steps: FlowStep[] }
  | { template: 'boxes'; title: string; boxes: Node[]; steps: BoxesStep[] }
  | { template: 'table'; title: string; columns: [string, string]; steps: TableStep[] }
  | { template: 'letters'; title: string; text: string; steps: LettersStep[] }
  | { template: 'compare'; title: string; leftLabel: string; rightLabel: string; steps: CompareStep[] };
```

The data for these 18 pictures goes in a **new file**,

`src/lib/data/lessonVisuals/pythonMonth1Visuals.ts`. It exports `PYTHON_M1_VISUALS: Record<string, { partTitle: string; visual: LessonVisual }>`, keyed `'python:<day>:<partIndex>'` (part index

from 0). The lesson file itself is not changed for visuals. `partTitle` must equal the lesson part's title, so a test catches any reordering.

C4. The voice moves the picture

Today, `useLessonEngine.ts` (around line 488) joins the whole part into one long `speech` string, so there is no moment when the picture could change.

Change:

- Keep the same words in the same order, but give the slide a list of pieces instead of one string: `{ at: 'intro' }` (the "Part N: title." sentence), then `say1...sayN`, then `example`, then `tryIt`, skipping empty ones.
- Speak the pieces one after another, using the existing per-utterance `onEnd` in `src/lib/tts.ts`. When a piece starts, call `onPiece(at)`, and the picture jumps to the last step whose `at` is at or before that piece.
- **Stop must stop the whole chain.** It must not start the next piece, and it must not fake a completion callback (the `CLAUDE.md` rule: `activeOnEndCallback = null`).
- Pause, Resume and Replay keep working. Replay restarts at the intro, with the picture at step 1.
- If the student presses Back or Next on the picture, voice sync for that part switches off until they press "Sync with voice" (a small text button under the picture). The student's choice wins.
- Without voice (muted, or TTS failed), the picture starts at step 1 and the student uses Next.

C5. Words and shapes point at each other (no markup in the lesson text)

- Do **not** write `<mark>` tags or any other markup into lesson text. The voice could read markup aloud, and showing HTML from data is a security risk.
- **Automatic matching instead:** for the current part, take the label of every node, box, `leftLabel` and `rightLabel`. In the right-hand text, wrap only the **first** whole-word, case-insensitive match **in each paragraph** in a `<button>` built by React. Underlining every match makes the text look messy (seen in the first screenshots: "balance" was underlined about 12 times). Never use `dangerouslySetInnerHTML`.
- Tapping or hovering a word sets the matching shape's tone to `data` for as long as it is hovered, or for 2 seconds after a tap. Tapping a shape does the same to its words in the text.
- Only `flow` and `boxes` shapes are tappable, and only when `tappable` is not `false`. Tables, `letters` and `compare` have no tappable shapes (in 1.4 both panels are about the word print, so tapping would light both). For those parts, nothing in the text is underlined.

C6. Rules, enforced by a test (`tests/lesson_visuals.test.ts`)

The test fails the build if any visual breaks any rule:

1. **Steps:** 2 to 5 per visual.
2. **Shapes:** at most 6 nodes, boxes or table rows. A `compare` has exactly 2 panels. `letters.text` has at most 10 characters.

3. **Order:** each `at` exists in the part (`sayN` only if the part has at least `N` `say` lines; `example` and `tryIt` only if the part has them), and each step's `at` comes strictly after the previous step's in the order `intro`, `say1...sayN`, `example`, `tryIt`.
4. **Caption:** one sentence, at most 80 characters, ending in `.`.
5. **Labels:** every tappable node and box label appears, case-insensitively and as a whole word, in that part's `say`, `example`, `code`, `codeNotes` or `tryIt`.
6. **Truth:** for every step, each string in `checks` must appear in the real output:
 - without `whatIf`, in the output of the part's `code`;
 - with `whatIf`, in the output of running that `whatIf` code.

Run the code with the same `runLikeLessonPage` (Pyodide) used by `tests/python_long_lessons.test.ts`. A check must equal one whole output line. A check ending in `Error` only needs to appear inside the `[Error] ...` line. `mustNotShow` values must not appear anywhere, and `lastLine` must equal the last line. 7. **Colours:** every tone is one of `data`, `ok`, `error`, `idle`. 8. **Match:** every key `python:<day>:<i>` exists, and `partTitle` equals the lesson part's title. 9. **Coverage:** all 18 parts of Days 1 to 3 have a visual, and no other key exists in this file.

C7. Layout and avatar

- **Desktop (1024 px and wider):** `LessonContentRenderer.tsx` today puts the standing avatar in `.interactive-left-col`. Replace that column's content with the visual stage. Use a grid of `minmax(0, 45fr) minmax(0, 55fr)` with `min-width: 0` on both columns.
- **Avatar:** move it to a bottom-right frame (`position: fixed; right: 24px; bottom: 24px;` about 220×165 px) with a minimise button that shrinks it to a 48 px circle. It must never cover the Run Code button or the quiz buttons, so add bottom padding to the right column equal to the frame height.
- **One avatar only:** the lesson page must show exactly one avatar. If the global floating avatar (`CLAUDE.md`, "Floating Avatar") also appears on `/quests/lesson`, hide it there. Add one line to `CLAUDE.md`: "On the lesson page the avatar is a bottom-right frame; the floating-avatar rule applies to the rest of the app."
- **No button may ever sit under the avatar frame**, including the bottom "Next Slide" button. Check at 1280, 1440 and 1920 px.
- **Under 1024 px:** one column. The visual stage comes first (collapsible, at most 260 px high), then the lesson. The avatar is the 48 px bubble.
- **A part without a visual:** the right column takes the full width. Days 1 to 3 have a visual in every part, so this only matters for other courses.

C8. Must NOT

- No video, GIF, image files, canvas or 3D, and no animation or diagram library.
- No looping or auto-playing motion. Nothing moves except on a step change.
- No new pictures, steps or values beyond Part D. If a step looks wrong, report it to Claude; do not change it yourself.
- No changes to any other course, or to Days 4 to 30.
- No `any` types, and no `dangerouslySetInnerHTML`.

- No redrawing from the student's own code in this pilot. That waits until the code-runner security fixes (audit F-tasks) are merged.

Part D. The 18 pictures, step by step

How to read this part:

- **at:** the spoken piece that starts the step.
- **checks:** values that the test confirms against real Python output.
- **whatIf:** the step shows the result of changed code (usually the "Your turn" change), and the test runs that code.
- **Tones:** if no tone is given, the tone is `data` for the step's new item, `ok` for results, and `idle` for everything else.

Day 1: Your First Python Program

1.1 What Python is and why so many people learn it: `flow`

Title: "From your instructions to the screen"

Nodes:

- `instructions` "Instructions"
- `python` "Python"
- `screen` "Screen"

Steps:

#	at	Shows	Caption
1	say1	<code>instructions = print("Hello! This is my first Python program.")</code> (data). Other boxes empty	You write exact instructions for the computer.
2	say2	arrow <code>instructions</code> → <code>python</code> lit; <code>python</code> = "reads it" (data)	Python reads your instructions, one line at a time.
3	example	arrow <code>python</code> → <code>screen</code> lit; <code>screen</code> = <code>Hello! This is my first Python program.</code> (ok)	The computer follows the steps and shows the result.

Checks (step 3): `Hello! This is my first Python program.`

1.2 `print()`: showing results on the screen: `table`

Title: "Each print line makes one screen line"

Columns: "Line", "Screen"

Steps:

#	at	Rows	Caption
1	say1	<code>print("My Expense Tracker")</code> → <code>My Expense Tracker</code> (ok)	<code>print</code> shows what is inside the brackets.

#	at	Rows	Caption
2	say3	adds <code>print("Tea", 20) → Tea 20</code> , <code>print('Bus ticket', 45) → Bus ticket 45</code> , <code>print(20 + 45) → 65</code> (all ok)	Four lines run top to bottom, so four results appear in order.
3	say4	same 4 rows; row 2 tone data	A comma prints both things with one space between.

Checks: step 1: `My Expense Tracker`; steps 2 and 3: `Tea 20`, `Bus ticket 45`, `65`.

1.3 How Python reads your code: line by line: `table` (needs fix B1 first)

Title: "Python reads top to bottom"

Columns: "Line", "What happens"

Steps:

#	at	Rows	Caption
1	say1	line 1 → Step 1: open the app (ok); line 2 → Step 2: add an expense (ok); line 3 → Step 3: see the total (ok)	Each line finishes before the next one starts.
2	say2	<code>print("Step 1: open the app") → Step 1: open the app (ok)</code> ; <code>prnt("Step 2: add an expense") → NameError (error)</code> ; <code>print("Step 3: see the total") → never ran (idle)</code>	A mistake while running stops Python; lines below never run.
3	say4	line 1 → nothing printed (idle); <code>print("Step 2: add an expense) → SyntaxError (error)</code> ; line 3 → nothing printed (idle)	A missing quote is a SyntaxError, so nothing runs at all.

Checks:

- step 1: the three `Step ...` lines;
- step 2: `whatIf` = the part's code with line 2 changed to `prnt("Step 2: add an expense")`; checks `Step 1: open the app`, `NameError`;
- step 3: `whatIf` = the part's code with line 2's closing quote removed; checks `SyntaxError`.

Step 3 also has `mustNotShow: ['Step 1: open the app']`. This proves the lesson's new sentence: nothing runs.

1.4 Capital letters matter: `compare`

Title: "Small and capital letters are different"

Panels: leftLabel "print", rightLabel "Print"

Steps:

#	at	Left / Right	Caption
1	say1	left <code>print("hello") → hello (ok)</code> . Right <code>Print("hello") → NameError (error)</code>	Python knows print, not Print.
2	say3	left <code>print("HELLO") → HELLO (ok)</code> . Right <code>print("Hello" == "hello") → False (data)</code>	Inside quotes any letters work, but H and h are still different.

Checks:

- step 1: left `hello`; right `whatIf = Print("hello")`, checks `NameError`;
- step 2: left `HELLO`; right `False`.

1.5 Comments: notes for humans: table**Title:** "Python skips everything after #"**Columns:** "Line", "What happens"**Steps:**

#	at	Rows	Caption
1	say1	# My Expense Tracker, day 1 → skipped (idle); <code>print("Tea", 20)</code> → Tea 20 (ok); <code># print("Movie", 300)</code> → (blank, idle); <code>print("Bus", 45)</code> # the bus to college → Bus 45 (ok)	A # line is a note for people; Python skips it.
2	say3	same rows; row 3 → turned off (data)	Putting # in front of a line turns it off without deleting it.

Checks: both steps: `Tea 20`, `Bus 45`. Step 2 also has `mustNotShow: ['Movie']`.**1.6 Putting it together: your first small program: table****Title:** "Your first receipt"**Columns:** "Line", "Screen"**Steps:**

#	at	Rows	Caption
1	say2	<code>print("=== My Expense Tracker ===")</code> → <code>=== My Expense Tracker</code> <code>===</code> ; <code>print("Tea", 20)</code> → Tea 20; <code>print("Bus", 45)</code> → Bus 45; <code>print("Lunch", 120)</code> → Lunch 120; <code>print("Total:", 20 + 45 + 120)</code> → Total: 185 (all ok)	A title, one line per expense, and the total at the bottom.
2	say3	same rows; rows 2 to 5 tone error	20, 45 and 120 are typed twice; tomorrow variables fix this.

Checks: both steps: `=== My Expense Tracker ===`, `Tea 20`, `Bus 45`, `Lunch 120`, `Total: 185`.**Day 2: Variables and Data Types****2.1 What a variable is: boxes****Title:** "A variable is a labelled box"**Boxes:**

- `tea` "tea"
- `bus_fare` "bus_fare"

- `lunch` "lunch"
- `total` "total"

Steps:

#	at	Values	Caption
1	say2	tea 20 (data); others empty	tea = 20 stores 20 in the box named tea.
2	say3	tea 20 , bus_fare 45 , lunch 120 (data)	= puts the value on the right into the name on the left.
3	example	adds total 185 (ok)	total uses the labels: 20 + 45 + 120 = 185.
4	tryIt	lunch 150 (data), total 215 (ok)	Change one box and the total follows: 215.

Checks:

- step 3: `Total: 185;`
- step 4: `whatIf` = the code with `lunch = 150;` checks `Total: 215.`

2.2 Changing a variable: boxes

Title: "A box keeps only its newest value"

Boxes: `balance` "balance"

Steps:

#	at	Values	Caption
1	say1	balance 500 (data)	balance starts at 500.
2	say2	balance 480 (data)	balance = balance - 20 stores 480; 500 is forgotten.
3	say3	balance 435 (data)	balance -= 45 is the short form: now 435.
4	tryIt	balance 1435 (ok)	balance += 1000 adds the salary: 1435.

Checks:

- step 1: `Start: 500;`
- step 2: `After tea: 480;`
- step 3: `After bus: 435;`
- step 4: `whatIf` = the code plus `balance += 1000` and `print("After salary:", balance);` checks `After salary: 1435.`

2.3 Types of values: text, whole numbers, decimals: boxes

Title: "Every value has a type"

Boxes:

- `item` "item"
- `amount` "amount"
- `rating` "rating"
- `twenty` "\"20\""

Steps:

#	at	Values (type tag)	Caption
1	say2	item "Tea" (str), amount 20 (int), rating 4.5 (float)	Text is str, whole numbers are int, decimals are float.
2	say3	same, with type tags tone ok	type() tells you the type: str, int or float.
3	say4	adds twenty "20" (str), tone error	Quotes make it text, even when it looks like a number.

Checks:

- step 2: `<class 'str'>`, `<class 'int'>`, `<class 'float'>`;
- step 3: `<class 'str'>`.

Mark `twenty` `tappable: false`, because "20" also appears in `amount = 20`.

2.4 True and False: the bool type: table (needs fix B2 first)

Title: "Questions answered True or False"

Columns: "Question", "Answer"

Steps:

#	at	Rows	Caption
1	say2	<code>amount > 1000</code> → True (ok); <code>amount == 500</code> → False (idle)	A question about a value gives True or False.
2	say3	<code>adds amount != 500</code> → True (ok); <code>is_big = amount >= 1000</code> → True (ok)	<code>!=</code> means not equal; <code>>=</code> means more than or equal.
3	tryIt	<code>amount 800</code> : False (data), False (idle), True (ok), False (data)	With 800, two answers change and two stay the same.

Checks:

- steps 1 and 2: the output lines `True`, `False`, `True`, `Big expense? True`;
- step 3: `whatIf` = the code with `amount = 800`; checks `Big expense? False`.

2.5 Converting between types: flow

Title: "Changing a value's type"

Nodes:

- `value` "value"
- `tool` "`str()` / `int()`"
- `result` "result"

Steps:

#	at	Shows	Caption
1	say2	value 20 (int), tool <code>str()</code> , result "Tea costs 20" (ok)	<code>str(20)</code> makes the text "20", so it can join other text.
2	say3	value "45" (str), tool <code>int()</code> , result <code>45 + 5 = 50</code> (ok)	<code>int("45")</code> makes the number 45, so maths works.
3	say4	value "abc", tool <code>int()</code> , result <code>ValueError</code> (error)	Text that is not a number cannot become a number.

Labels: these node labels are general words, not names from the code. Mark all three nodes `tappable: false`.

Checks:

- step 1: `Tea costs 20`;
- step 2: `50`;
- step 3: `whatIf = int("abc"); checks ValueError`.

2.6 Good variable names and a better tracker: boxes

Title: "The tracker with variables"

Boxes:

- `monthly_budget` "monthly_budget"
- `tea` "tea"
- `bus_fare` "bus_fare"
- `lunch` "lunch"
- `total` "total"
- `left` "left"

Steps:

#	at	Values	Caption
1	say1	5000, 20, 45, 120, total 185 (ok), left 4815 (ok)	Each amount is stored once; total and left are worked out.
2	tryIt	<code>monthly_budget 100</code> (data), total 185, left -85 (error)	With a budget of 100 the money left is negative.

Checks:

- step 1: `Spent today: 185, Left this month: 4815, Over budget? False`;
- step 2: `whatIf` with `monthly_budget = 100`; checks `Left this month: -85, Over budget? True`.

Day 3: Working With Text

3.1 Strings and joining them: flow

Title: "+" joins text exactly as it is"

Nodes:

- `a` "Hello, "
- `b` "first"
- `c` "!"
- `greeting` "greeting"

Steps:

#	at	Shows	Caption
1	say2	a Hello, , b Asha, c ! (data); arrows a→b→c lit; greeting Hello, Asha! (ok)	+ joins in order; the space must be inside the quotes.
2	say3	a - , b * 20, c empty, greeting ----- - (ok)	* repeats a string: one dash, 20 times.

Checks: step 1: Hello, Asha! ; step 2: ----- .

Labels: "Hello, " and "!" are code text. Mark `a` and `c` `tappable: false` .

3.2 Length and positions: letters

Title: "Positions start at 0"

Text: Priya

Steps:

#	at	Pointer / range	Result	Caption
1	say1	range [0, 5)	<code>len(name) = 5</code> (ok)	len() counts every character: 5.
2	say2	pointer 0	<code>name[0] = P</code> (data)	The first character is at position 0, not 1.
3	say3	pointer -1	<code>name[-1] = a</code> (data)	-1 is always the last character.
4	say4	pointer 10 (drawn after the end)	IndexError (error)	There is nothing at position 10, so Python gives an IndexError.

Checks:

- step 1: 5;
- step 2: P;
- step 3: a;
- step 4: `whatIf = name = "Priya" then print(name[10]);` checks `IndexError` .

Under each cell, draw the positive index in normal text and the negative index (-5 to -1) in muted text.

3.3 Slicing: taking part of a string: letters

Title: "A slice stops before stop"

Text: 2026-09-28 (exactly 10 characters, the maximum)

Steps:

#	at	Range	Result	Caption
1	say2	[0, 4); cell 4 outlined idle	date[0:4] = 2026 (ok)	Positions 0 to 3 are taken; position 4 is not.
2	say3	[8, 10)	date[-2:] = 28 (ok)	Leave out stop to go to the end: the last two characters.
3	say4	none; all cells data	date is still 2026-09-28 (data)	Slicing gives a new string; the original stays the same.

Checks: step 1: 2026 ; step 2: 28 .

3.4 String tools: upper, lower, strip, replace: flow

Title: "Cleaning text step by step"

Nodes:

- typed "typed"
- strip "strip()"
- lower "lower()"
- category "category"

Steps:

#	at	Shows	Caption
1	say2	typed FOOD , lower lit, category food (ok); strip idle	lower() turns FOOD into food, so all spellings match.
2	say3	typed ··FOOD· , arrows typed→strip→lower→category lit, category food (ok)	strip() removes the spaces first, then lower() runs.
3	say4	typed ··FOOD· (data), category food (ok); no arrows	typed is unchanged; the clean text is stored in category.

Checks: step 2: [food] .

3.5 Searching inside text: table

Title: "Asking questions about text"

Columns: "Question", "Answer"

Steps:

#	at	Rows	Caption
1	say1	"card" in note → True (ok); "cash" in note → False (idle)	in checks whether the smaller text is inside the bigger one.
2	say2	adds "invoice.pdf".endswith(".pdf") → True (ok)	endswith() checks the end, startswith() the beginning.

#	at	Rows	Caption
3	say3	adds <code>"banana".count("a") → 3</code> (ok); <code>note.find("team") → 11</code> (ok)	<code>count()</code> counts copies; <code>find()</code> gives the first position.
4	say4	adds <code>"Card"</code> in <code>note</code> → <code>False</code> (error)	Capital C is different, so <code>"Card"</code> is not found.

Checks:

- steps 1 to 3: `True`, `False`, `3`, `11`;
- step 4: `whatIf` = the code plus `print("Card" in note)`, with `lastLine: 'False'`.

3.6 Putting it together: tidy expense labels: flow

Title: "Clean the input, then use it"

Nodes:

- `item` "item"
- `clean_item` "clean_item"
- `category` "category"
- `clean_category` "clean_category"
- `label` "label"

The node `label` is the printed line: mark it `tappable: false`.

Steps:

#	at	Shows	Caption
1	say1	item <code>·masala·CHAI·</code> (error), category <code>·FOOD</code> (error); others empty	Typed text is messy: extra spaces and random capitals.
2	say2	arrows <code>item→clean_item</code> , <code>category→clean_category</code> , both→ <code>label</code> lit; <code>clean_item</code> <code>Masala chai</code> , <code>clean_category</code> <code>food</code> , <code>label</code> <code>Masala chai (food)</code> (ok)	<code>strip()</code> , <code>capitalize()</code> and <code>lower()</code> clean it, then <code>+</code> builds the label.

Checks: step 2: `Masala chai (food)`.

Part E. Tasks for Antigravity (in this order)

ID	Task	Files	Done when
V-01	Fix the two lesson mistakes in Part B, word for word	<code>src/lib/data/pythonLongLessons.ts</code>	<code>npm test</code> passes; the Day 1 Part 3 and Day 2 Part 4 text matches Part B
V-02	Add the data types from C3	<code>src/lib/types/lessonVisual.ts</code> (new)	<code>npx tsc --noEmit clean</code>
V-03	Write the rule test C6, with one deliberately broken sample per rule to prove each rule fails	<code>tests/lesson_visuals.test.ts</code> (new)	Each broken sample fails with a clear message

ID	Task	Files	Done when
V-04	Enter the 18 visuals exactly as in Part D	<code>src/lib/data/lessonVisuals/pythonMonth1Visuals.ts</code> (new)	V-03 passes on the real data
V-05	Build the 5 templates and a <code>VisualStage</code> (title, picture, caption, Back, Next, step dots, "Sync with voice")	<code>src/app/quests/lesson/components/visuals/</code> (new folder)	Each template renders every step of its Part D visuals with no console errors
V-06	Pass <code>visual</code> into the slide, and speech as pieces (C4)	<code>src/app/quests/lesson/hooks/useLessonEngine.ts</code> , <code>src/lib/tts.ts</code> (only if needed)	The picture changes on the right sentence; Stop never starts the next piece
V-07	Word ↔ shape tapping (C5)	<code>src/app/quests/lesson/components/LessonContentRenderer.tsx</code>	Tapping <code>balance</code> in 2.2 lights the box, and back
V-08	Layout and avatar (C7), plus the one-line <code>CLAUDE.md</code> note	<code>LessonContentRenderer.tsx</code> , the lesson page CSS, <code>CLAUDE.md</code>	Desktop 45/55 with bottom-right avatar; phone stacked; one avatar only
V-09	Full check	—	<code>node</code> <code>claude_self_validate.js</code> passes; <code>npm test</code> passes; the test prints <code>lesson visuals: all checks passed</code> ; screenshots of 1.3, 2.2, 3.2 and 3.4 at 1440 px and 390 px wide, in light and dark mode, sent to the owner

Branch rules: work on `main-dis3ku`, and merge to `main` only through a pull request whose "Web app tests" check is green. Never push to `main` directly.

Part F. What Claude will check when Antigravity says "done"

1. Part B text is changed exactly, and nothing else in the lesson file changed.
2. Part D data matches this document, item by item: steps, `at`, captions and values.
3. The rule test fails when a value is changed on purpose (for example `435` → `436` in 2.2).
4. There is no `dangerouslySetInnerHTML`, no `any`, no animation library in `package.json`, and no `@keyframes` with `infinite`.
5. Stop in the middle of a part: no further sentence is spoken, and the picture stays where it was.
6. Reduced motion is on: there are no transitions.
7. Screenshots match A1 at both widths.